UNIT 5

- **Input / Output:**
Peripheral devices, I/O interface, I/O ports

Interrupts:
interrupt hardware, types of interrupts and exceptions.

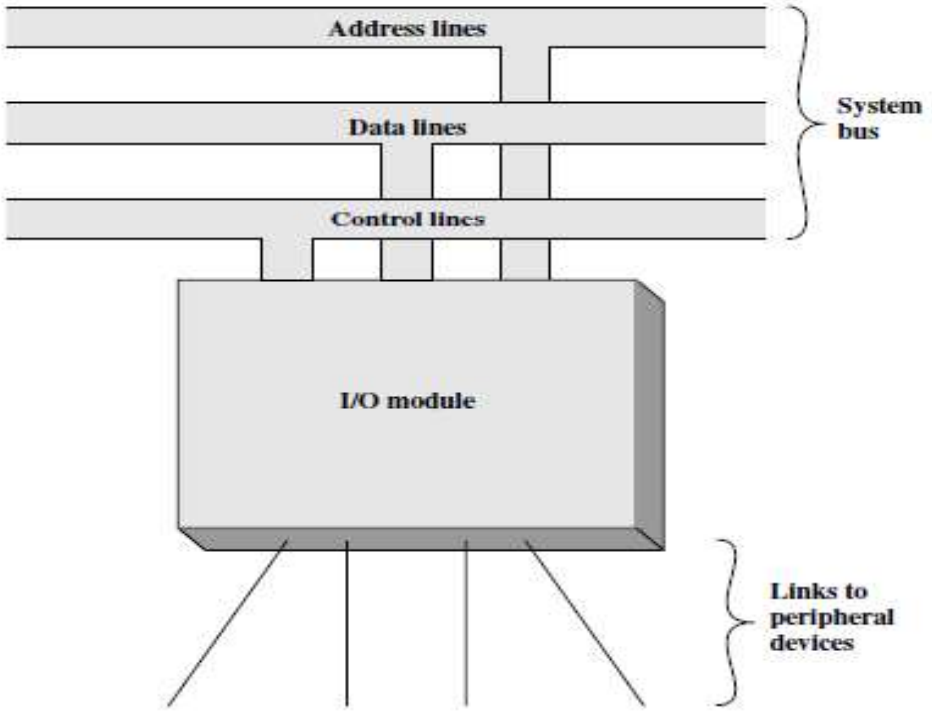
- **Modes of Data Transfer:**
Programmed I/O, interrupt initiated I/O and Direct Memory Access., I/O channels and processors.

- **Serial Communication:** Synchronous & asynchronous communication, standard communication interfaces.

I/O Module

- Generic Model

- I/O Module interfaces to the system bus or central switch to control peripheral devices.
- It contains logic for performing a communication function between the peripheral and system bus



[1]

I/O Module

- Peripheral Device
- Introduction
- Classification of external devices.

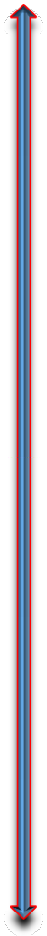
- An external device connected to a I/O module is often referred to peripheral device.
- Peripheral devices helps users to access and use the functionality of computer system. E.g. keyboard, mouse, printer etc.
- External device is attached to computer through a link to I/O module, this link is used to exchange control, status and data bits.

There are three classification of external devices:

1. Human Readable: Monitors, printers
2. Machine readable: tapes, sensors, actuators
3. Communication devices: other computer, or remote machine readable device

I/O Module

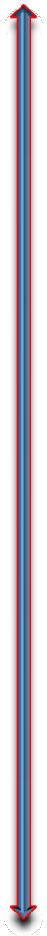
- Need of Separate I/O Module



- There are a wide variety of peripherals making it impractical to incorporate the necessary logic within the processor
- The data transfer rate of peripherals is often much slower than that of the memory or processor. Synchronization is required.
- Peripherals often use different data formats and word lengths than the computer to which they are attached.

I/O Module

- Functions of I/O Module

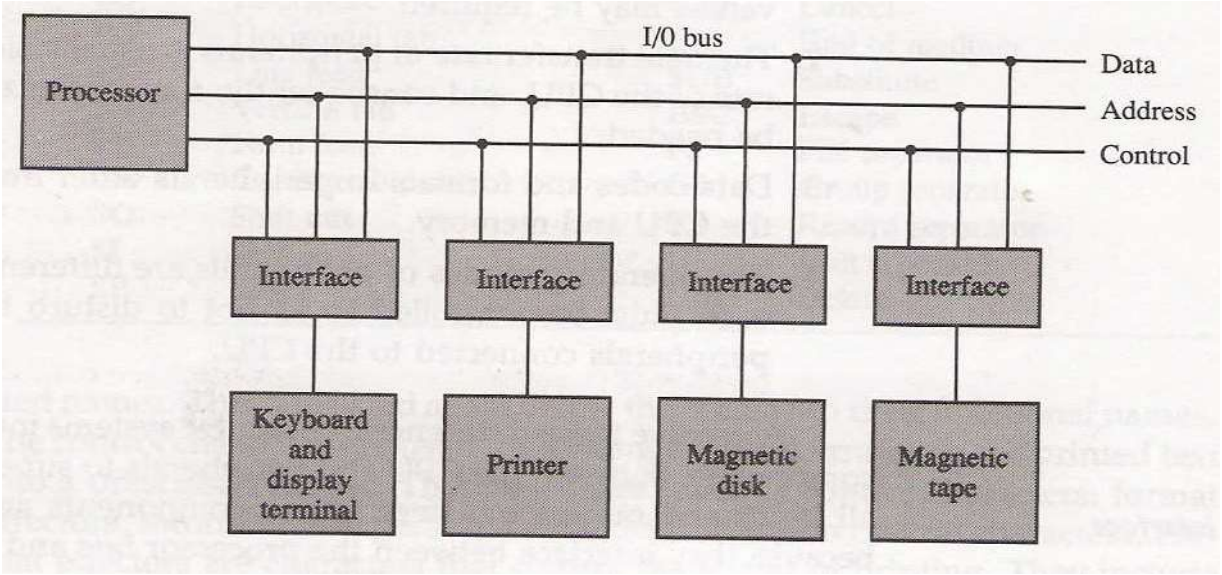


- Control & Timing
- CPU Communication
- Device Communication
- Data Buffering
- Error Detection

I/O Module

- I/O Interface

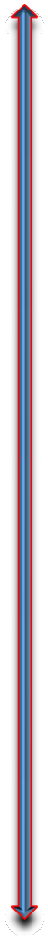
•Interface supervises and synchronize all input/output transfers
•Each peripheral device uses interface unit from I/O Module
•Interface Decode address and control received from I/O
•To communicate processor put device address on address lines, Each interface monitors address lines, if matched activates path.



[2]

I/O Module

- I/O Interface(2)



•All devices with non matching address are disabled through corresponding interface.

• The selected interface respond to function code and is referred to as I/O Command

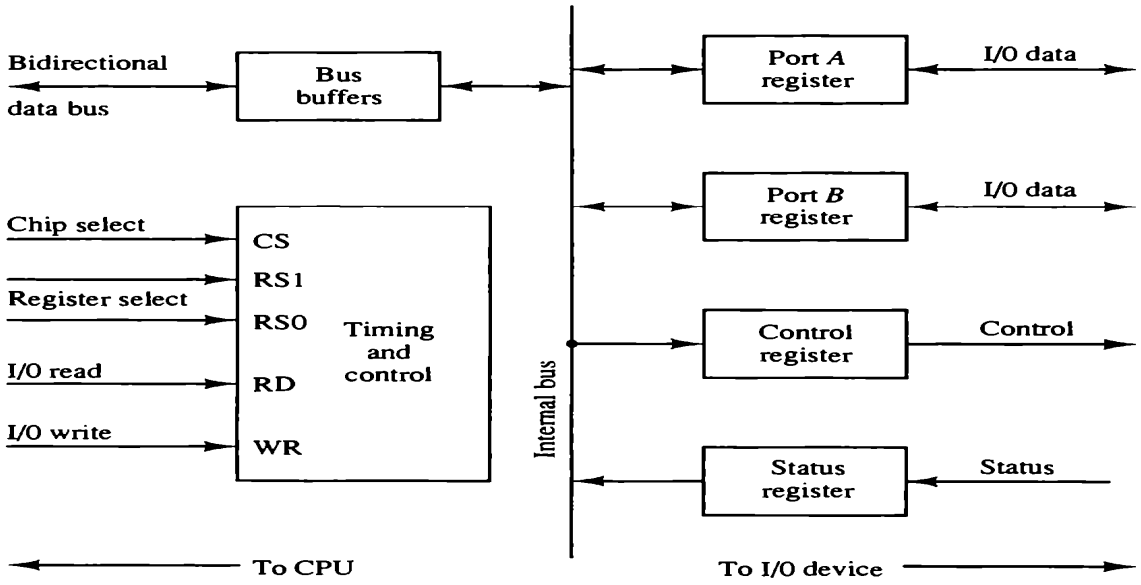
•Interpretation of command depends on peripheral
• There could be four types of commands

1. Control: Used to activate a peripheral and tell it what to do
2. Test/Status: Used to test various status conditions
3. Read: Enables the I/O module to obtain data from peripheral
4. Write: Enables the I/O module to take data (byte or word) from the data bus and transmit that data item to the peripheral

[2]

I/O Module

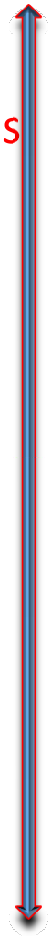
- I/O Interface Example



CS	RS1	RS0	Register selected
0	×	×	None: data bus in high–impedance
1	0	0	Port *A* register
1	0	1	Port *B* register
1	1	0	Control register
1	1	1	Status register

I/O Module

- I/O vs Memory Bus

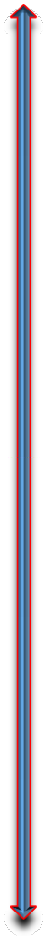


There are three methods which can used in computer system for Memory or I/O transfer

1. Use separate buses memory and I/O
2. Use common buses with separate control line for each
3. Use one common bus for memory and I/O transfer with common control lines.(Mostly Used)

I/O Module

- I/O Ports



➢A connection point that act as interface between computer and external devices.

Types:

1. Internal Ports: Used to connect internal devices like hard disk drive, CD Drive etc. with motherboard
2. External Ports: Used to connect motherboard to external devices like mouse, printer, flash drive etc.

Input output channel and processors

- I/O module with enhance capability of I/O execution are termed as I/O channel.
- I/O Channel greatly reduced the Main processor burden as I/O operations are performed through it without much CPU involvement
- A special processor in I/O channel execute I/O instructions.

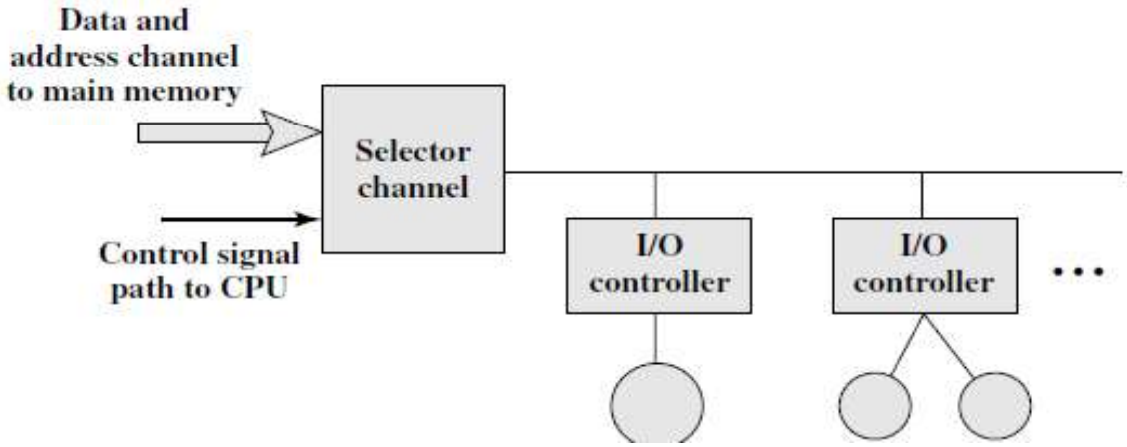
Types of I/O channel

1. Selector
2. A Multiplexor Channel

I/O channel

- Selector Channel

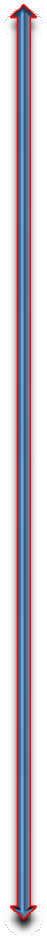
- Controls multiple high speed devices & uses one at a time
- I/O channel controls the corresponding I/O control at place of CPU



[1]

I/O channel

- Multiplexor Channel



- Can handle number of devices at a time.
- For slow devices byte multiplexing is used
- E.g. if we have three devices having following stream of bytes

B1, B2, B3….. Data from device 1

C1,C2,C3…….. Data from Device-2

D1,D2,D3……. Data from Device-3

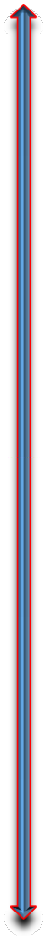
In this case byte multiplexer will send data as

B1C1D1B2C2D2……

[1]

❖ Introduction - Interrupts

- A signal to processor by hardware/Software stating immediate attention
- Kind of high priority alert to CPU to interrupt its ongoing activity
- In this case CPU stores current state, and handles the interrupt through handler
- Once the interrupt is handled CPU resumes its normal execution
- Actually most of the operation are being performed through interrupts e.g. using keyboard, mouse, playing any file



❖ Types of Interrupts

❑ Hardware Interrupts such as pressing key from the keyboard

a) Maskable(Lower priority)

b) Non Maskable (Higher Priority)

❑ Software Interrupts such as arithmetic overflow, division by zero, illegal machine instruction

a) Normal interrupts: Through Software instructions

b) Exceptions: divide by zero (unplanned)

❖ Further Classification

❑Based on Period

- Periodic (e.g. periodic polling after fix interval)
- Unpredictable

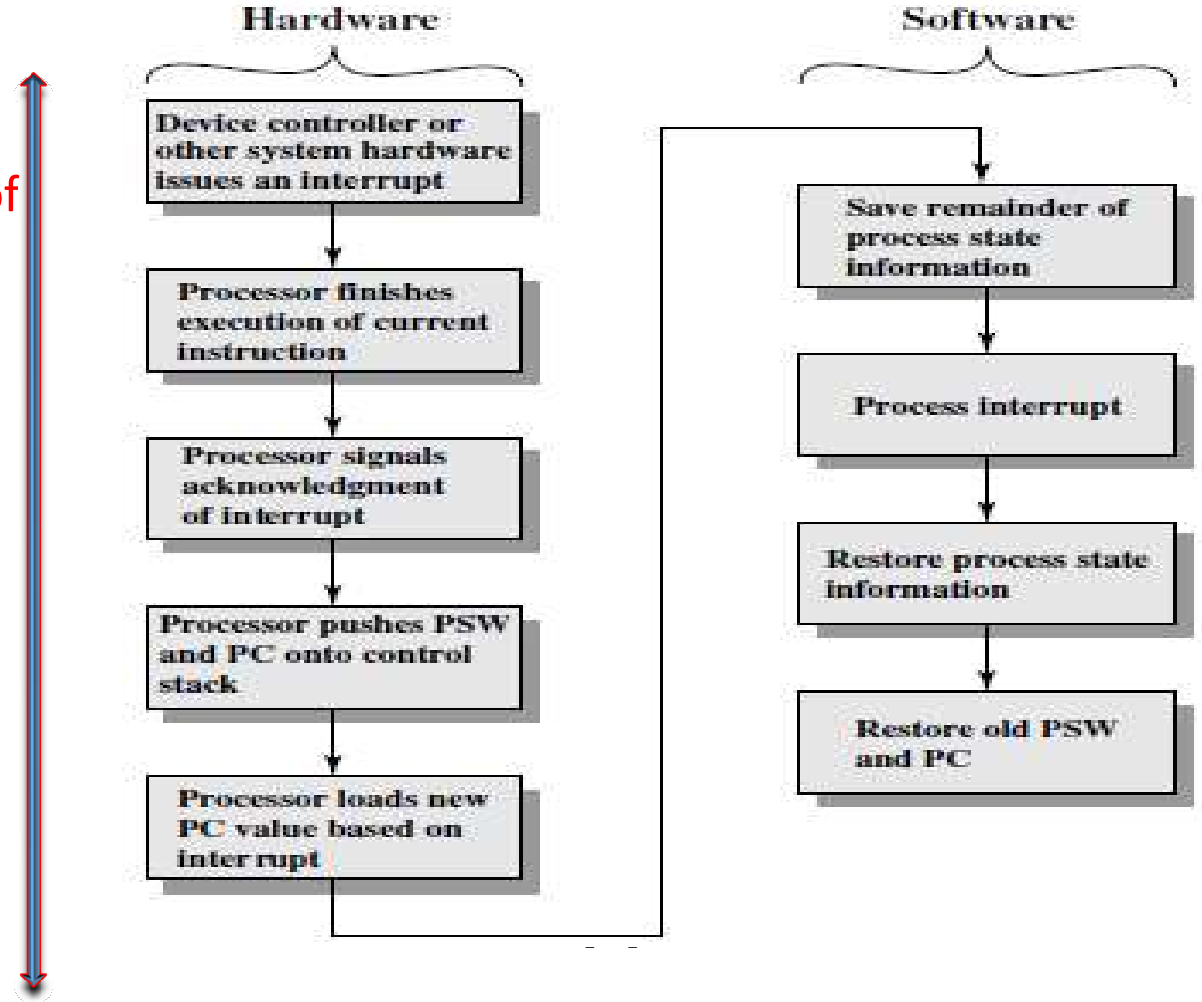
❑Based on time

- Asynchronous (Independent of clock)
- Synchronous (Based on clock pulse)

❑ Based on CPU & Address Availability

- **Vectored**(ISR is known in advance through bus and I/O interface, CPU Check IVT and execute corresponding ISR.
- **Non Vectored**(No ISR is sent by interrupting device, CPU Jumps to specific address in hardware, to know status each device needs to be polled.

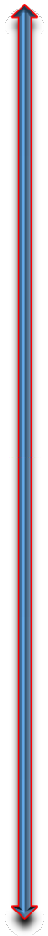
- Processing cycle of Interrupt



❖ Interrupt

• Selection

➢If multiple devices interrupts at same time then Certain arbitration mechanism is used by CPU



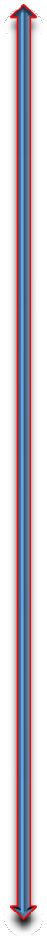
a) CPU with Shared interrupt request Line

- Bus arbitration technique like daisy chain is used
- All devices are scanned on receiving interrupt signal by scanning corresponding ports

a) CPU with independent interrupt request line

- Easily identifies corresponding interrupting device.
- Priority mechanism is used in this case
- This approach waste many lines of system bus for initiating interrupt

Modes of Data Transfer



- Data transfer between central computer and I/O can be handled in different ways:

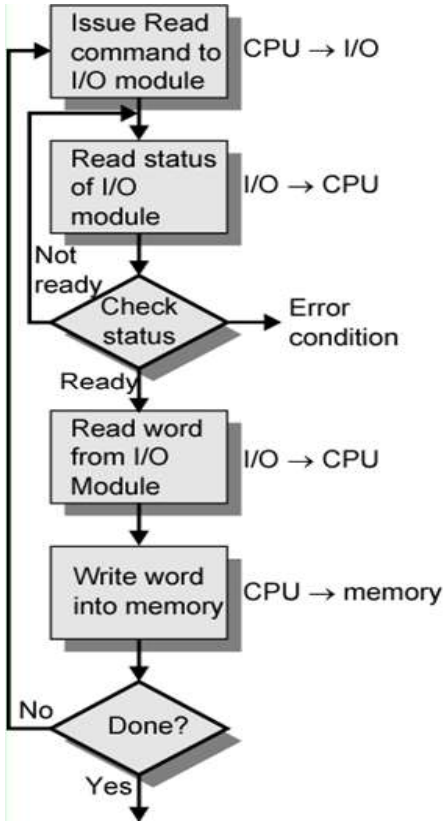
1. Programmed I/O
2. Interrupt Initiated I/O
3. Direct Memory Access

In these techniques CPU may / May not be involved partially or completely

Programmed I/O

- Concept
- Process

- CPU has direct control over I/O
  1. Sensing status
  2. Read/write commands
  3. Transferring data
- CPU waits for I/O module to complete operation
- Wastes CPU time
- CPU requests I/O operation
- I/O module performs operation
- I/O module sets status bits
- CPU checks status bits periodically
- I/O module does not inform CPU directly
- I/O module does not interrupt CPU
- CPU may wait or come back later



Programmed I/O

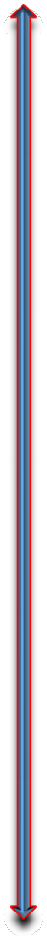
- Memory Mapped I/O
- Isolated I/O

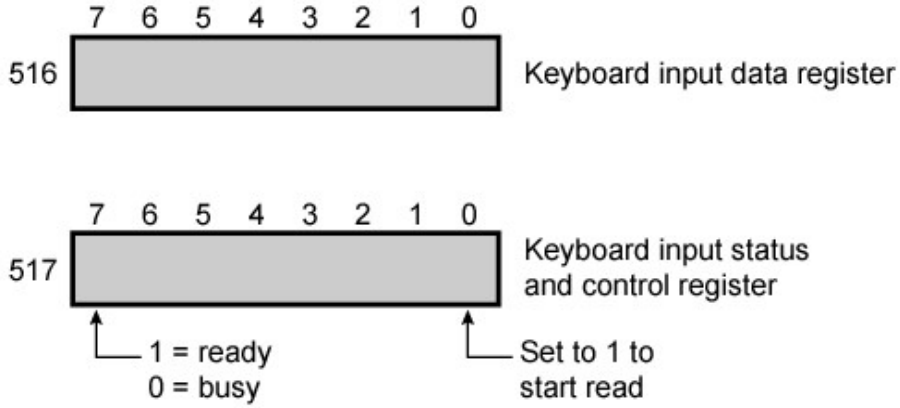
In shared bus scenario where CPU, memory and IO share same Bus two types of addressing are used

- Memory mapped I/O
  - Devices and memory share same address space
  - Processor Uses the same machine instructions to access both memory and I/O devices
  - I/O looks just like memory read/write
  - No special commands for I/O
- Isolated I/O
  - Separate address spaces for I/O and Memory Devices
  - Need I/O or memory select lines with address
  - Special commands for I/O

Programmed I/O

- Memory Mapped vs Isolated I/O
- Example





ADDRESS	INSTRUCTION	OPERAND	COMMENT
200	Load AC	"1"	Load accumulator
	Store AC	517	Initiate keyboard read
202	Load AC	517	Get status byte
	Branch if Sign = 0	202	Loop until ready
	Load AC	516	Load data byte

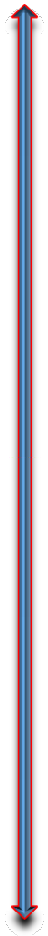
(a) Memory-mapped I/O

ADDRESS	INSTRUCTION	OPERAND	COMMENT
200	Load I/O	5	Initiate keyboard read
201	Test I/O	5	Check for completion
	Branch Not Ready	201	Loop until complete
	In	5	Load data byte

(b) Isolated I/O

Interrupt Driven I/O

- Introduction
- Basic Operation



- Overcomes CPU waiting
- No repeated CPU checking of device
- I/O module interrupts when ready

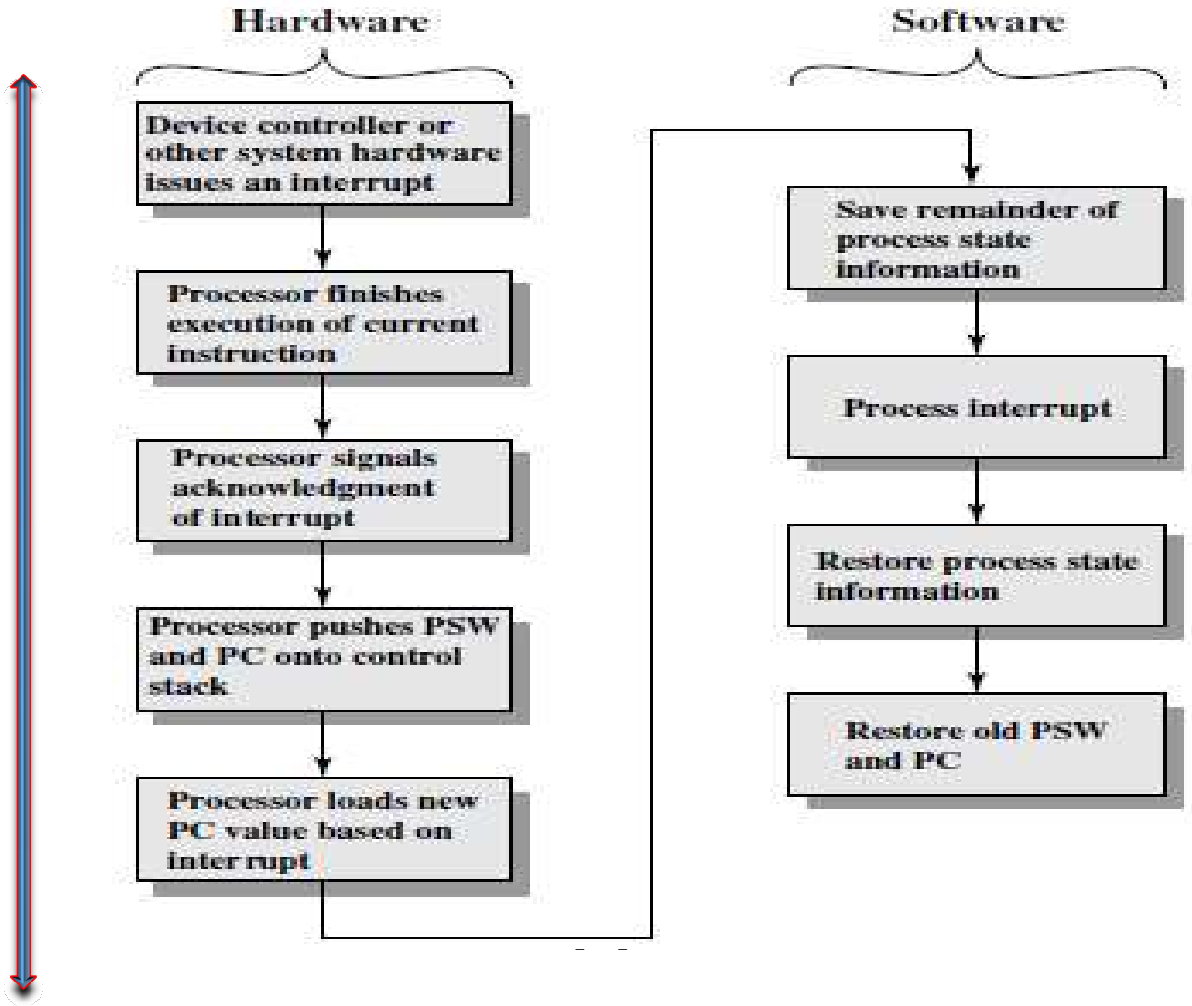
- CPU issues read command to I/O module
- I/O module gets data from peripheral whilst CPU does other work
- I/O module interrupts CPU
- CPU requests data
- I/O module transfers data

Interrupt Driven I/O

- Basic Process from CPU point of view

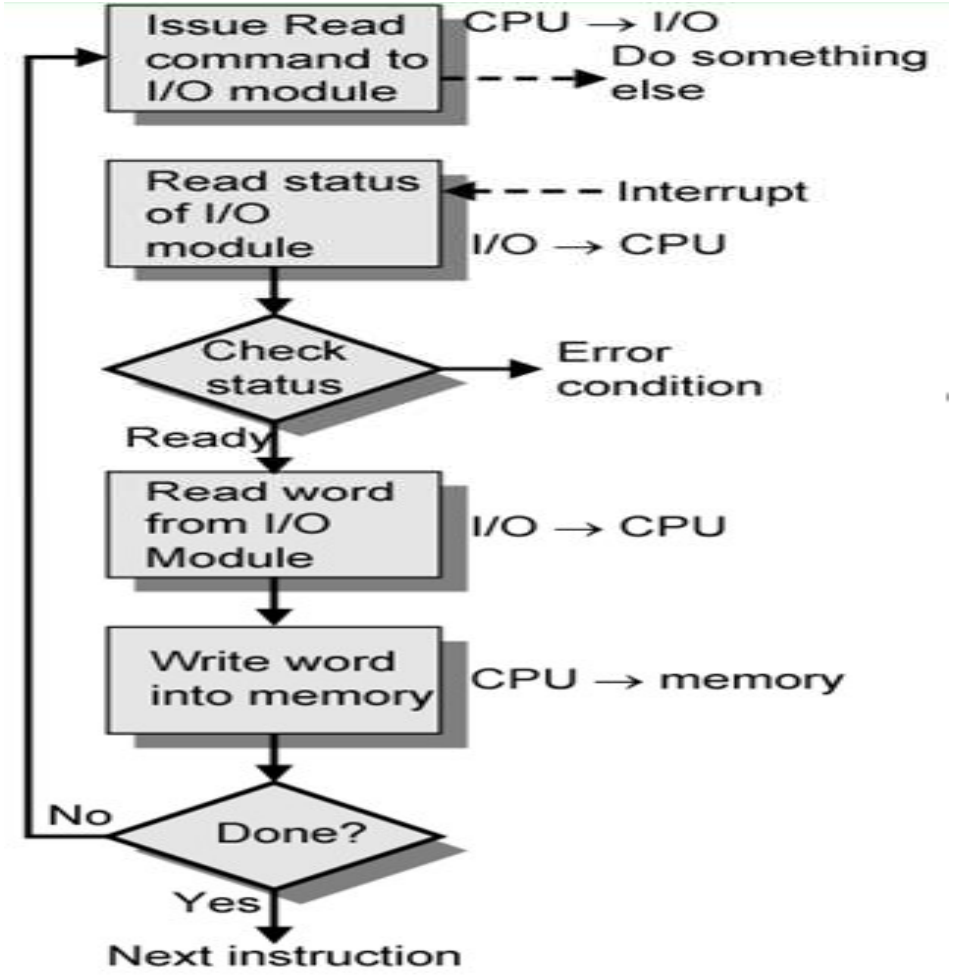
- Issue read command
- Do other work
- Check for interrupt at end of each instruction cycle
- If interrupted:-
  - Save context (registers)
  - Process interrupt
    - Fetch data & store

Interrupt Driven I/O



Interrupted Driven I/O

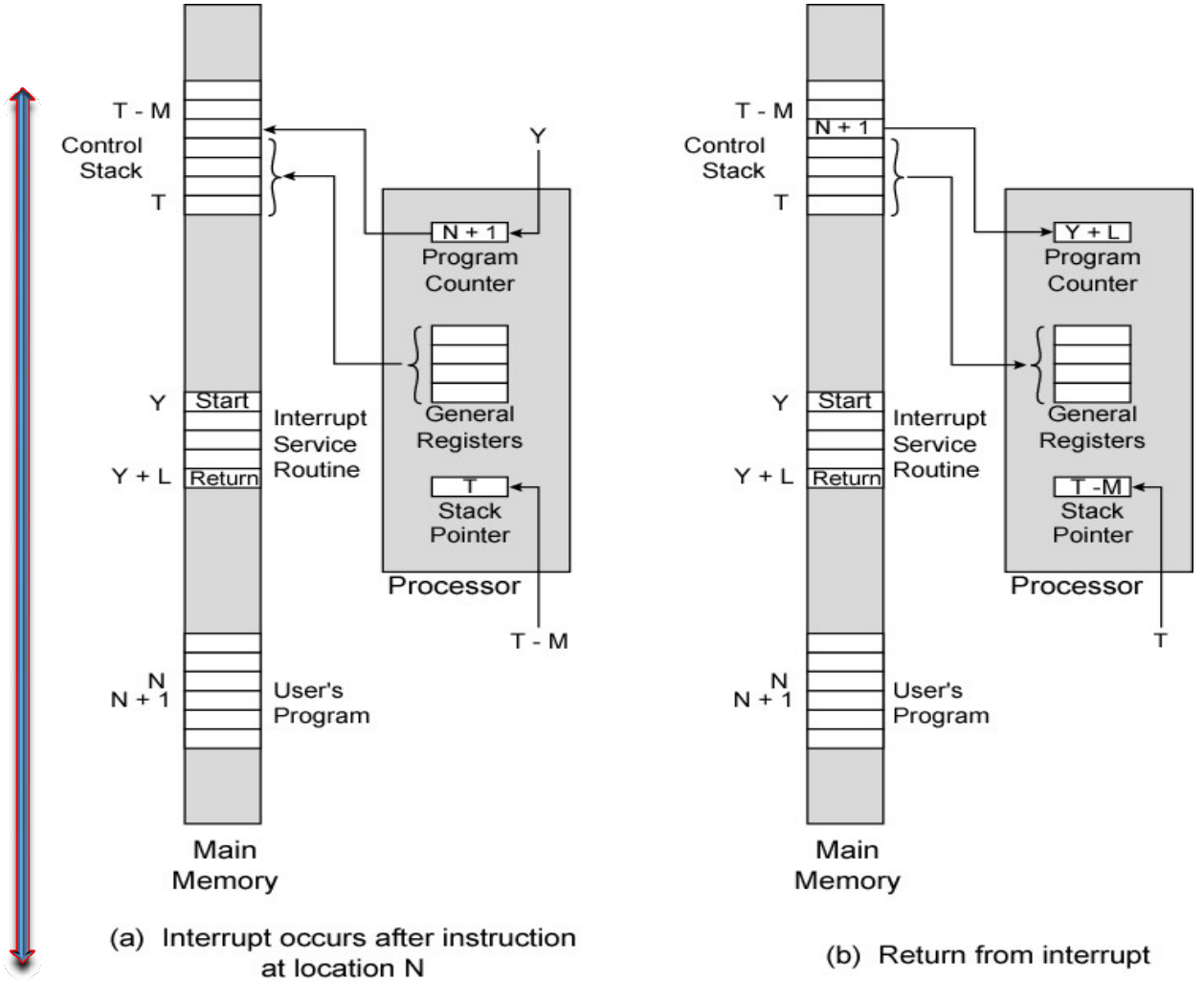
- Basic Process



[1]

Interrupted Driven I/O

- Changes in Memory and Registers during Interrupt



[1]

Priority Interrupt

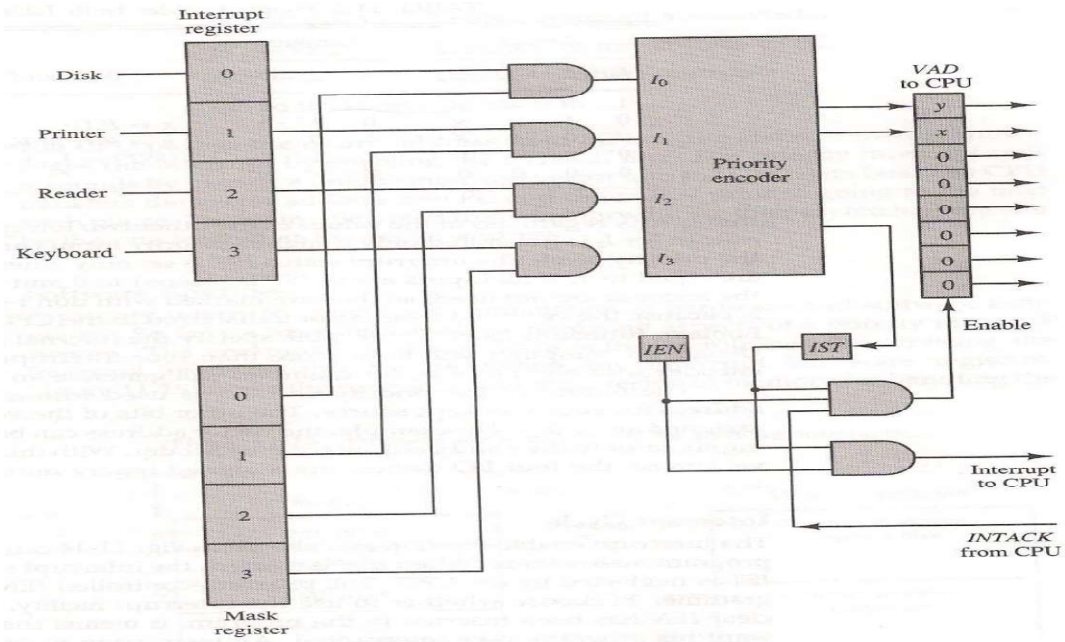
❖ An example

- A Number of devices are attached to computer each having ability to interrupt.
- Interrupt controller will first identify source/sources of interrupt
- System must decide which one to serve first.
- Assigning Priority can be one of the solution
- Devices with high speed transfers are given high priority and slow devices are given low priority.
- Priority can be maintained at hardware(Daisy chaining/Parallel Priority) or software level.(Polling)

Priority Interrupt

- Hardware implementation

➢Priority Interrupt Hardware



Inputs				Outputs		
I_0	I_1	I_2	I_3	x	y	IST
1	X	X	X	0	0	1
0	1	X	X	0	1	1
0	0	1	X	1	0	1
0	0	0	1	1	1	1
0	0	0	0	X	X	0

Summary

- A Comparison

Programmed I/O	Interrupt Driven I/O
It can be implemented without any additional Hardware circuit	Additional hardware is required to handle interrupt
It is simple and used in low-end system where cost is important factor	Most of high-end systems are interrupt driven where speed is important factor
Busy waiting is involved	CPU switches to other jobs while I/O is going on
Only one I/O Can be handled at time	Multiple I/O activity can be handled in overlapped Fashion

DMA Transfer

- Introduction

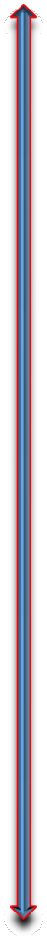
➢Third approach of I/O data Transfer

DMA: Direct Memory Transfer

- In transfer of data between storage device such as magnetic disc and memory if we are able to remove the CPU and let the peripheral manage the transfer directly would be called as DMA.
- During DMA transfer , CPU is idle and has no control of the buses. This control is transferred to DMA controller.

DMA

❖ Making CPU Idle



Can be done through two control signals

- Bus Request : Used by DMA controller to inform the CPU to relinquish the control of buses.
- Bus Grant: Is used to inform the DMA controller that request has been accepted and CPU has released the control of buses.

DMA

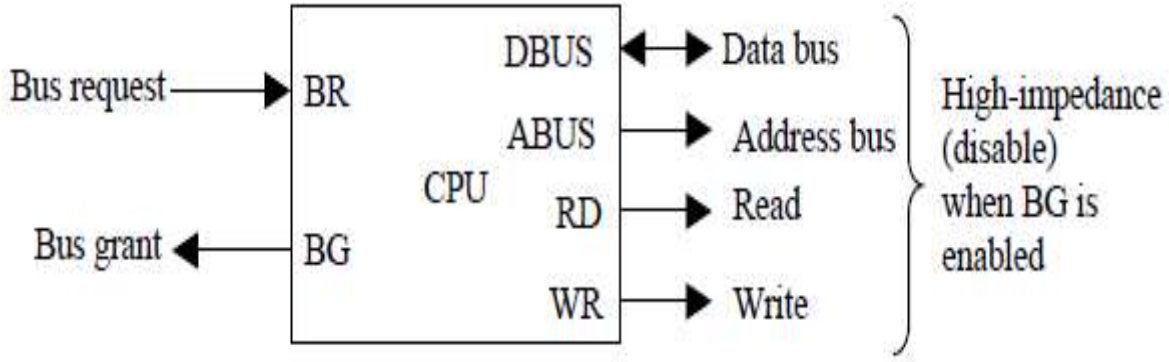
- Burst Transfer vs. Cycle stealing Transfer method

- When DMA takes control of Buses. It communicates directly with memory.
- Burst Transfer means a block sequence of memory words is transferred in continuous burst without giving control back to CPU.
- Cycle Stealing allows the DMA controller to transfer one data word at a time and then return control to CPU.
- CPU leaves one cycle intentionally free for each DMA Transfer alternatively

DMA

- CPU Bus signal for DMA transfer

- In addition to usual circuit it needs an address register, a word count register, and a set of address lines
- Word count register is used to mention number of words
- When the BG =0, the CPU can communicate with the DMA registers through the data bus to read/write DMA register
- When BG = 1, the CPU has relinquished the buses and the DMA can communicate directly with the memory by specifying an address in the address bus through activating the RD or WR control



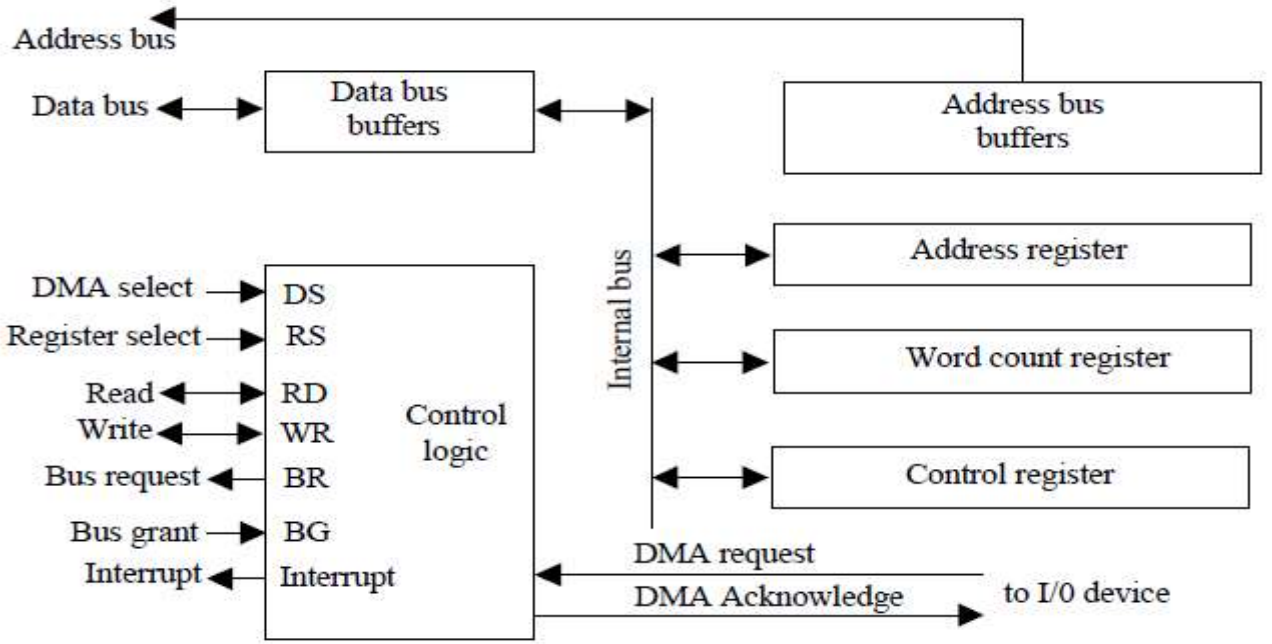
DMA

- Block Diagram of DMA Controller

•Register in DMA are selected by CPU by enabling DS and RS through address bus

•DMA communicates with peripheral through request & ack line using handshake procedure.

•DMA controller is initialized with starting address, word count, RD/WR control and DMA enable control



DMA Transfer

❖ Making CPU Idle

- First Peripheral device sends DMA request.
- DMA controller activates BR line.
- CPU respond with BG line indicating disabling of Bus by CPU.
- DMA puts the current value of AR into Add. Bus. And initiates RD or WR signal
- DMA Send DMA ack to peripheral
- After Receiving the DMA ack . Peripharal puts a word in data bus(for write) or read a word from data bus(for read).
- Now the peripheral unit can directly communicate memory through data bus till the CPU is momentarily disabled.
- For Each word that is transferred DMA Increments its address register and decrements its word count register.
- If the word count register reaches zero, the DMA stops any further tranfer and removes its bus request. And control of buses is taken back by CPU.

DMA

- DMA Transfer in computer system